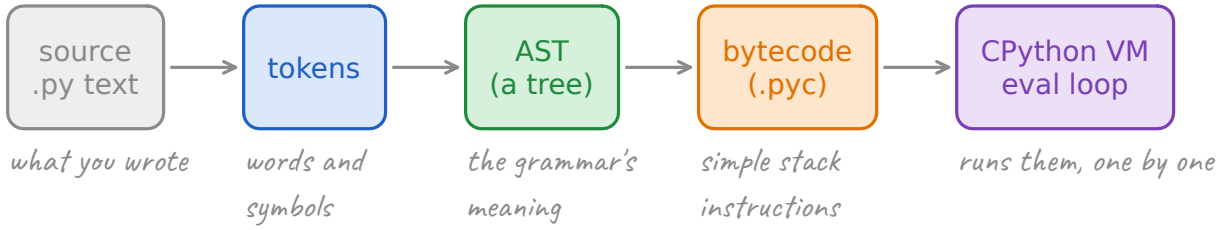
Chapter 2

How Python runs your code

from the text you type to things happening

When you run `python3 grades.py`, Python doesn't read your file line by line and act on it. It **translates the whole file first**, then runs the translation on a virtual machine. Knowing the stages explains a lot of everyday puzzles: why a syntax error stops everything before line 1 runs, why a misspelt name only fails when that line is reached, and what those `__pycache__` folders are.



stage	done by	you can look at it with
tokenise	the tokenizer	`python3 -m tokenize file.py`
parse	the parser	`ast.parse(src)`, `python3 -m ast file.py`
compile	the compiler	`compile(src, name, 'exec')`, `dis.dis(func)`
execute	the evaluation loop (ceval.c)	`sys.settrace`, `pdb`, or just `print`

KEY IDEA

Compile time is when the whole file is checked and translated. **Run time** is when each instruction actually executes. Most errors you meet happen at run time; syntax errors are the exception.

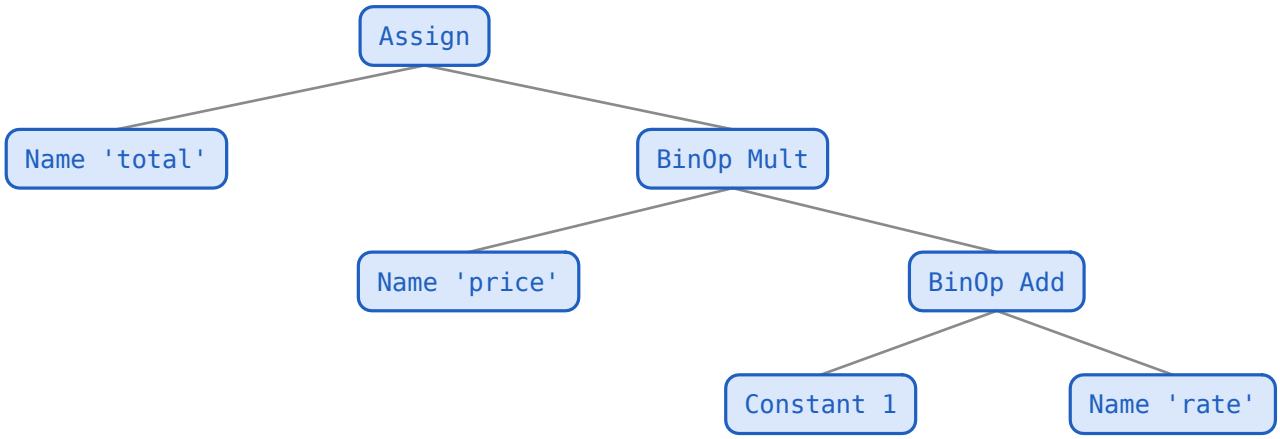
Stage 1 and 2: tokens, then a tree

The tokenizer chops text into tokens: names, operators, numbers, and indentation changes (Python's blocks are real tokens, `INDENT` and `DEDENT`).

```
import io, tokenize
src = "total = price * (1 + rate)"
for tok in tokenize.generate_tokens(io.StringIO(src).readline):
    if tok.string.strip():
        print(f"{tokenize.tok_name[tok.type]:<6} {tok.string!r}")
```

```
NAME   'total'
OP     '='
NAME   'price'
OP     '*'
OP     '('
NUMBER '1'
OP     '+'
NAME   'rate'
OP     ')'
```

The parser checks the tokens against Python's grammar and builds an **abstract syntax tree**. Brackets have gone: the tree's shape **is** the precedence. Multiplication sits above the addition, so the addition happens first:




```
>>> import ast
>>> print(ast.dump(ast.parse("x = a + b * 2").body[0].value))
BinOp(left=Name(id='a', ctx=Load()), op=Add(), right=BinOp(left=Name(id='b', ctx=Load…
```

> **WHY**
> This is where `SyntaxError` comes from: if the tokens can't form a tree, there's no program to run. That's why one missing bracket on line 200 stops line 1 from running.

Stage 3: bytecode, for a stack machine

The compiler turns the tree into **bytecode**: a list of tiny instructions for a virtual machine that works with a stack. Values are pushed on, operations pop their inputs and push the result. `dis` shows it:

```python
import dis

def f(a, b):
    return a * (b + 1)

for ins in dis.get_instructions(f):
    print(f"{ins.opname:<36} {ins.argrepr}")
```

```
RESUME
LOAD_FAST_BORROW_LOAD_FAST_BORROW    a, b
LOAD_SMALL_INT
BINARY_OP                            +
BINARY_OP                            *
RETURN_VALUE
```

Here's the stack as `f(3, 4)` runs (the long first name is two loads fused into one instruction, a 3.14 speed trick):

instruction	stack afterwards (top on the right)
`LOAD_FAST..._LOAD_FAST (a, b)`	`3  4`
`LOAD_SMALL_INT 1`	`3  4  1`
`BINARY_OP +`	`3  5`
`BINARY_OP *`	`15`
`RETURN_VALUE`	`(empty) → returns 15`

IN OTHER LANGUAGES

This is the same design as the Java Virtual Machine and .NET: compile to bytecode, run it on a VM. C and Rust compile straight to machine code instead, which is a big part of why they're faster for tight loops. PyPy is a different Python implementation that compiles hot bytecode to machine code while running (a JIT); CPython 3.13+ has an experimental JIT too.

Compile time vs run time

Two programs, each with one mistake on its last line. Watch **when** each mistake is noticed:

```
src = 'print("line 1 runs?")\nprint("oops"'
try:
    compile(src, "a.py", "exec")
except SyntaxError as e:
    print("SyntaxError before anything ran:", e.msg, "(line", e.lineno, ")")
```

```
SyntaxError before anything ran: '(' was never closed (line 2 )
```

```
src = 'print("line 1 runs?")\nprint(undefined_name)'
try:
    exec(compile(src, "b.py", "exec"))
except NameError as e:
    print("NameError, after line 1 ran:", e)
```

```
line 1 runs?
NameError, after line 1 ran: name 'undefined_name' is not defined
```

The first program never started. The second ran line 1 and only failed when it reached the bad name, because looking a name up is a run-time action. The same goes for functions: a typo inside a function body is only noticed when the function is **called**.

```
def report():
    return totl * 2          # typo, but no error yet

print("defined fine")
try:
    report()
except NameError as e:
    print("only now:", e)
```

← *compiles: it's just a name*

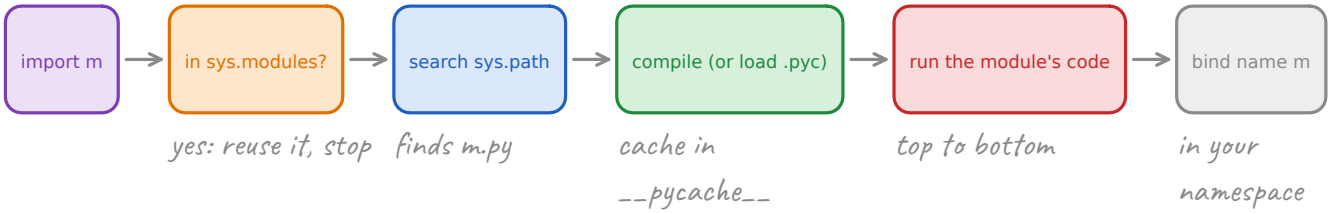
```
defined fine
only now: name 'totl' is not defined
```

TRAP

So "it ran without errors" means only the paths you exercised are free of these mistakes. Tests (Ch 17) and a linter or type checker (which read the code without running it) catch the rest.

What import really does

`import grades` isn't a text include. It's a run-time statement that **executes** the module once and caches the result:




```
1  import sys, pathlib, importlib
2
3  pathlib.Path("noisy.py").write_text('print("noisy is running!")\nX = 42\n')
4  sys.path.insert(0, ".")
5
6  import noisy
7  import noisy
8  print(noisy.X, "noisy" in sys.modules)
9  print(sorted(p.name for p in pathlib.Path("__pycache__").iterdir()))
10 importlib.reload(noisy)
```

```
noisy is running!
42 True
['noisy.cpython-314.pyc']
noisy is running!
```

line 6: runs the file

line 7: cached: prints nothing

line 9: the saved bytecode

line 10: reload forces a re-run

Because importing **runs** the file, any top-level code (prints, tests, a whole program) runs on import. The standard guard keeps script-only code from running when the file is imported:

```
def main():
    print("doing the real work")

if __name__ == "__main__":     # true only when run as a script
    main()
```

how the file is used	its `__name__` is	so the guarded code...
`python3 grades.py`	`"__main__"`	runs
`python3 -m grades`	`"__main__"`	runs
`import grades` (from another file)	`"grades"`	doesn't run

TRAP

Never name your file after a stdlib module (`random.py`, `json.py`, `secrets.py`). The current folder comes first on `sys.path`, so `import random` anywhere in your program imports **your** file instead.

Ways to start Python, and which Python

```
$ python3 grades.py              # run a file
$ python3 -m http.server 8000    # run a module found on sys.path (here: a stdlib web se
$ python3 -c 'print(2**100)'     # run a string
$ python3 -i grades.py           # run, then drop into the REPL with its variables
$ python3 -X importtime -c 'import json'   # how long each import took
```

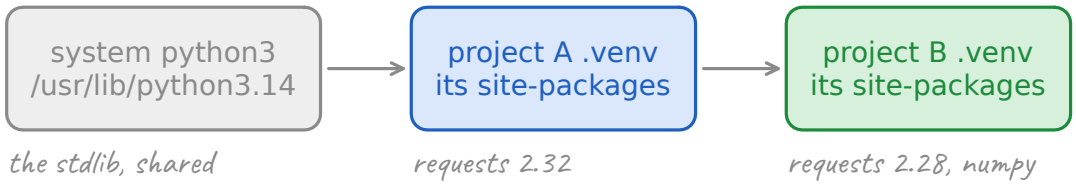
You may have several Pythons installed. Inside a program, `sys` tells you which one is running and where it looks for modules:

```python
import sys
print(sys.version_info[:3], sys.implementation.name)
print(sys.executable.rsplit("/", 2)[-2:])
print([p for p in sys.path if p.endswith("site-packages")][:1])
```

```
(3, 14, 6) cpython
['bin', 'python3']
['/home/Marau/.local/lib/python3.14/site-packages']
```

Virtual environments in one picture

A virtual environment (`python3 -m venv .venv`) is a folder with its own `python` link and its own `site-packages`, so each project's pip installs stay separate. Activating it just puts its `bin/` first on your `PATH`. Chapter 19 uses one.



Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 2.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 2.1` shows other valid solutions and their trade-offs.

`2.1` learn an unseen module Count the node types in a program (ast)
`2.2` build from a spec A safe calculator (no eval)
`2.3` learn an unseen module Which globals does a function use? (dis)
`2.4` refactor, tests stay green Make the script importable
`2.5` mini-project Mini-project: compile to your own stack machine

PASSPORT STAMP ask the same question of every new language (Appendix B)

1. How does code run?

CPython compiles each module to bytecode (cached as `.pyc` in `__pycache__`), then a stack-based VM interprets it. Syntax errors at compile time; name, type and value errors at run time.

9. Modules and build

`import` runs a module once and caches it in `sys.modules`; it's found on `sys.path`. `if __name__ == "__main__"` separates script code from library code.

DEEP DIVE (your vault)

Vault notes: 15.2 From source bytes to an abstract syntax tree · 15.4 Code objects · 15.5 Disassembly and bytecode · 15.8 The evaluation loop · 10.2 Import is execution with caching · 10.5 `__name__` and entry execution · 10.14 Import security and shadowing.